```
$ redis-cli ping
```

The response you should see is `PONG`.

- You can avoid the use of redis entirely in development by ensuring the `development` section of the `config/cable.yml` is as follows:

```
development:
  adapter: async
```

What We Just Did

In this iteration, we added Hotwire support to our cart:

- We moved the shopping cart into the sidebar. We then arranged for the `create` action to redisplay the catalog page.
- We used `as: :turbo_stream` to indicate to the `LineItemsController.create` that the client supports Turbo Streams.
- We then used an ERB partial template to return only the portions of the page that need to be replaced.
- We used Action Cable and Turbo Frames to update the catalog display whenever a product changes.
- We wrote a test that verifies not only the creation of a line item but also the content of the response that's returned from such a request.

The key point to take away is the incremental style of Hotwired development. Start with a conventional application and then add Ajax features, one by one. Hotwired applications can be hard to debug; by adding it slowly to an application, you make it easier to track down what changed if your application stops working. And, as you saw, starting with a conventional application makes it easier to support both Hotwired and non-Hotwired behavior in the same codebase.

Finally, here are a couple of hints. First, if you plan to do a lot of Hotwired